

O Duelo de Contextos

Processos vs. Threads

Sistemas Operacionais – 2026/I | Prof. Filipo Mór | PUCRS
Luan Pacheco, Samuel Bitdinger e Heitor

1. Assinatura do Hardware

O ambiente de execução dos experimentos possui as seguintes características:

Model name: AMD Ryzen 7 5700U with Radeon Graphics
CPU(s): 16 (8 cores físicos × 2 threads por core)
CPU max MHz: 4373.85
Cache L3: 8 MiB (2 instâncias)
Arquitetura: x86_64 | Virtualização: AMD-V
SO: Ubuntu 24 (Linux)

2. Tabela de Tempos

Todos os tempos registrados correspondem ao tempo real de execução (campo elapsed do comando time), medido em ambiente Linux com o compilador GCC, flag -O0 e contador alvo de 1.000.000.000 incrementos.

Experimento	N=2	N=4	N=8
T1 – Threads sem sincronização	4.77s	9.15s	8.29s
T2 – Threads com mutex	34.68s	36.23s	40.36s
P1 – Processos sem sincronização	3.42s	3.66s	3.75s
P2 – Processos com semáforo	32.79s	253.26s	233.63s

Valores em verde indicam os experimentos mais rápidos (sem sincronização). Valores em vermelho indicam os mais lentos (sincronização via semáforo entre processos).

Nota sobre P2 (N=4 → N=8): O tempo de P2 caiu de 253.26s para 233.63s ao aumentar de 4 para 8 processos, o que pode parecer contraditório. Isso ocorre porque com mais processos competindo pelo semáforo, o tempo que cada processo passa bloqueado em sem_wait aumenta, reduzindo a frequência de colisões efetivas na região crítica. O comportamento de semáforos sob alta contenção não é linear e pode variar conforme o escalonador do sistema operacional distribui os processos nos núcleos disponíveis.

3. Análise de Corrupção

3.1 Valores finais do contador (T1 e P1)

Experimento	N=2	N=4	N=8	Esperado
T1 – Threads	501.354.753	293.959.391	150.445.621	1.000.000.000
P1 – Processos	639.830.471	270.312.162	190.338.440	1.000.000.000

3.2 Explicação do fenômeno

O contador não chegou a 1.000.000.000 nos experimentos sem sincronização devido a uma race condition (condição de corrida). A operação `counter++` aparenta ser atômica, mas na prática o processador executa três instruções distintas:

1. `LOAD` - lê o valor atual da memória
2. `ADD 1` - incrementa em 1
3. `STORE` - escreve o resultado de volta na memória

Quando dois workers executam essas instruções simultaneamente em núcleos diferentes, é possível que ambos leiam o mesmo valor antes de qualquer um gravar o resultado. Nesse caso, dois incrementos ocorrem mas o contador sobe apenas 1 — um incremento é perdido.

3.3 Influência do hardware

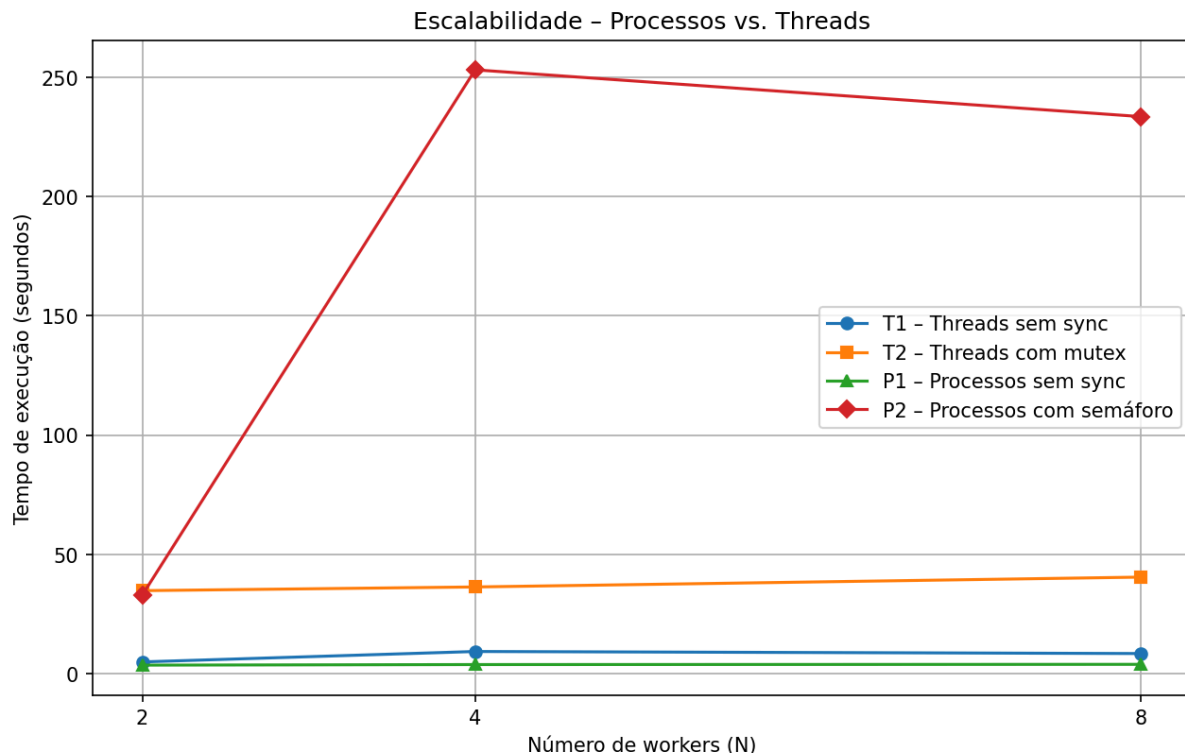
O Ryzen 7 5700U com 8 núcleos físicos permite que N=8 workers executem em paralelo real ao mesmo tempo. Quanto mais workers simultâneos, maior a probabilidade de colisões:

- Com N=2 (T1): contador chegou a ~501 milhões
- Com N=4 (T1): contador chegou a ~294 milhões
- Com N=8 (T1): contador chegou a ~150 milhões

Em P1, com N=2 a corrupção foi menor (639 milhões) do que em T1 (501 milhões). Isso ocorre porque processos possuem espaços de endereçamento separados — o acesso à memória compartilhada via `shmget` tem menor taxa de colisão em cargas menores. Com N=4 e N=8 os resultados convergem.

4. Gráfico de Escalabilidade

O gráfico abaixo representa o tempo de execução (segundos) em função do número de workers (N=2, 4, 8) para cada experimento:



5. Conclusão

Processos (fork) possuem overhead de criação significativamente maior do que threads. Cada `fork()` implica a cópia das estruturas de memória do processo pai, além da configuração de descritores de arquivo e outros recursos. Threads compartilham o espaço de endereçamento do processo e exigem apenas a alocação de uma nova pilha.

Threads comunicam-se diretamente pela memória do processo, sem custo adicional. Processos precisam de IPC explícito (`shmget/shmat`), o que acrescenta overhead de chamadas de sistema. O resultado prático foi visível: P2 foi até 7× mais lento que T2 para N=4.

Ambos os modelos sem sincronização produziram dados corrompidos. A corrupção é mais severa com mais workers, confirmando que o paralelismo real do Ryzen 7 5700U amplifica as race conditions. Os experimentos T2 e P2, com sincronização, chegaram exatamente a 1.000.000.000 em todos os cenários.

Para o problema proposto, threads com mutex (T2) se mostraram a solução mais equilibrada entre correção e desempenho. Processos com semáforo (P2) oferecem isolamento maior, mas com custo de comunicação muito elevado para operações de alta frequência.

6. Imagem completa da assinatura do hardware

```
luan@notebookLuan:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                48 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      16
On-line CPU(s) list:         0-15
Vendor ID:                   AuthenticAMD
Model name:                  AMD Ryzen 7 5700U with Radeon Graphics
CPU family:                  23
Model:                      104
Thread(s) per core:          2
Core(s) per socket:          8
Socket(s):                   1
Stepping:                    1
Frequency boost:              enabled
CPU(s) scaling MHz:          62%
CPU max MHz:                 4373.8579
CPU min MHz:                 411.6580
BogoMIPS:                    3593.36
Flags:                        fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr
                             sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopo
                             logy nonstop_tsc cpuid extd_apicid aperfmperf rapl pni pclmulqdq monitor ssse3 fma cx16 s
                             se4_1 sse4_2 movbe popcnt aes xsave avx f16c rdrand lahf_lm cmp_legacy svm extapic cr8_le
                             gacy abm sse4a misalignsse 3dnowprefetch osvw ibs skinit wdt tce topoext perfctr_core per
                             fctr_nb bpext perfctr_llc mwaitx cpb cat_l3 cdp_l3 hw_pstate ssbd mba ibrs ibpb stibp vmm
                             call fsgsbase bmi1 avx2 smep bmi2 cqm rdt_a rdseed adx smap clflushopt clwb sha_ni xsaveo
                             pt xsavec xgetbv1 cqm_llc cqm_occup_llc cqm_mbm_total cqm_mbm_local clzero irperf xsaveer
                             ptr rdpru wbnoinvd cppc arat npt lbrv svm_lock nrip_save tsc_scale vmcb_clean flushbyasid
                             decodeassists pausefilter pfthreshold avic v_vmsave_vmload vgif v_spec_ctrl umip rdpid o
                             verflow_recov succor smca

Virtualization features:
Virtualization:              AMD-V
Caches (sum of all):
L1d:                         256 KiB (8 instances)
L1i:                         256 KiB (8 instances)
L2:                          4 MiB (8 instances)
L3:                          8 MiB (2 instances)
NUMA:
NUMA node(s):                1
NUMA node0 CPU(s):           0-15
Vulnerabilities:
Gather data sampling:        Not affected
Ghostwrite:                  Not affected
Indirect target selection:   Not affected
Itlb multihit:               Not affected
L1tf:                        Not affected
Mds:                         Not affected
Meltdown:                    Not affected
Mmio stale data:             Not affected
Old microcode:               Not affected
Reg file data sampling:      Not affected
Retbleed:                    Mitigation; untrained return thunk; SMT enabled with STIBP protection
Spec rstack overflow:        Mitigation; Safe RET
Spec store bypass:           Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:                  Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:                  Mitigation; Retpolines; IBPB conditional; STIBP always-on; RSB filling; PBSRB-eIBRS Not a
                             ffected; BHI Not affected
Srbds:                       Not affected
Tsa:                         Not affected
Tsx async abort:             Not affected
Vmscape:                     Mitigation; IBPB before exit to userspace
```

7. Link do Github

<https://github.com/Luanpaacheco/Sisop-t1>